

Exercício PC01: Pesquisa não informado no jogo “8-puzzle”

Introdução

O objetivo deste trabalho é implementar e comparar algoritmos de pesquisa não informada para resolver o Jogo “8-Puzzle”. O objetivo final é analisar a solução encontrada e o uso de memória dos algoritmos de pesquisa em largura (BFS), pesquisa em profundidade (DFS) e pesquisa de custo uniforme (UCS).

O problema a resolver

O Jogo 8-puzzle consiste num tabuleiro 3x3 com as peças 1 a 8 e um espaço vazio.

A configuração de cada tabuleiro é representada por um *tuplo* constituído por 3 *tuplos*, cada um com 3 números, que representam cada linha do tabuleiro. **O espaço vazio é representado pelo dígito 0.**

Exemplo de estado: ((1, 2, 3), (4, 0, 5), (6, 7, 8))

O objetivo do problema é alcançar a configuração alvo representada por ((1, 2, 3), (4, 5, 6), (7, 8, 0)), que corresponde ao seguinte tabuleiro:

1	2	3
4	5	6
7	8	

Algoritmos a implementar e regras de custo

Devem ser implementados três métodos de pesquisa.

Cada ação consiste em mover uma peça adjacente ao espaço vazio, sendo possível mover cada peça para cima, baixo, esquerda ou direita, quando possível. Em alternativa, cada ação pode ser descrita como mover o espaço vazio uma posição para cima, baixo, esquerda ou direita, quando possível. Nos algoritmos BFS e DFS, cada movimento tem custo 1. No caso do algoritmo UCS deve utilizar os custos definidos na tabela

Resumo dos algoritmos a implementar:

1. **Pesquisa em Largura (BFS):** Custo 1 por passo.
2. **Pesquisa em Profundidade (DFS):** Custo 1 por passo.
Deve implementar DFS com controlo de estados repetidos para evitar ciclos infinitos.
3. **Pesquisa de Custo Uniforme (UCS):** O custo de **mover uma peça numerada para o espaço vazio** depende da direção do movimento da peça:
 - Mover peça para **CIMA**: Custo **2.0**
 - Mover peça para **BAIXO**: Custo **0.5**
 - Mover peça para a **ESQUERDA**: Custo **1.0**
 - Mover peça para a **DIREITA**: Custo **1.0**

Nota: Tenha em atenção que mover uma peça para cima corresponde a mover o espaço vazio para baixo.

Casos de Teste

Os algoritmos serão validados com os seguintes estados iniciais:

- **Teste A (simples):** ((1, 2, 3), (0, 4, 5), (7, 8, 6))
- **Teste B (médio):** ((0, 1, 6), (5, 3, 2), (4, 7, 8))
- **Teste C (mais complexo):** ((7, 2, 0), (5, 6, 4), (8, 3, 1))
- **Outros testes:** O código será testado com outras combinações para validar a generalização.

Normas de Entrega (Obrigatório)

Deverá ser entregue na plataforma Moodle um ficheiro que **tem de cumprir rigorosamente estas regras** (sob pena de não ser avaliado pela ferramenta de avaliação automática):

1. **Nome do Ficheiro:** **exsps_<numero>.py** em que numero corresponde ao número de aluno (ex: exsps_24268.py).
2. **Bibliotecas:** Apenas é permitida recorrer a **biblioteca standard** do Python (collections, heapq, etc.). O uso de outras bibliotecas como numpy, networkx, etc., ou outras bibliotecas externas resultará em erro de importação, não sendo o trabalho entregue avaliado.
3. **Estrutura do Código:** O ficheiro deve conter exatamente estas três funções:

- `bfs(start, goal)` -> retorna (caminho, custo_total, max_memoria)
- `dfs(start, goal)` -> retorna (caminho, custo_total, max_memoria)
- `ucs(start, goal)` -> retorna (caminho, custo_total, max_memoria)

em que:

- **caminho**: solução encontrada que deve ser uma lista de representações de tabuleiros (tuplos de tuplos). O caminho deve incluir o estado inicial e o estado final.

Exemplo:

```
[ ((1,2,3),(4,0,5),(6,7,8)),
  ((1,0,3),(4,2,5),(6,7,8)),
  ((1,2,3),(4,5,6),(7,8,0))
]
```

- **max_memoria**: corresponde ao maior número de estados que estiveram "em espera" para serem explorados, isto é o tamanho máximo da fronteira (fila, pilha ou fila de prioridade) durante a execução do algoritmo
- **custo_total**: custo do caminho da solução devolvida pelo algoritmo

4. As funções não devem imprimir resultados no ecrã.
Devem apenas devolver os valores indicados.

Template de Código para o Aluno

```
def bfs(start, goal):
    # A completar
    return caminho, custo_total, max_memoria

def dfs(start, goal):
    # A completar
    return caminho, custo_total, max_memoria

def ucs(start, goal):
    # A completar
    return caminho, custo_total, max_memoria
```

Auto-validação de código

Juntamente com este enunciado, é fornecido um ficheiro *validate.py* com um exemplo do código fonte que será utilizado para validar o ficheiro submetido pelo aluno. A disponibilização deste ficheiro permite realizar o teste do código antes de ser submetido no moodle (e assim evitar a entrega de código fonte com erros de formatação ou nomes de funções errados, etc.).

Para testar o código fonte escrito, deve colocar este ficheiro na mesma pasta que o seu ficheiro *exsps_<numero>.py* e executar o ficheiro *validate.py*.

Ficheiros fornecidos

- *validate.py*: Ficheiro python para auto-validação do código produzido
- *exsps_99999.py*: ficheiro python exemplo com os dados e respetivos tipos a retornar por cada função a implementar.

Recomendações

Ao gerar estados sucessores, não deve modificar diretamente o estado atual. Deve criar um novo estado com as alterações necessárias.

Os algoritmos devem manter um conjunto de estados já visitados, de forma a evitar explorar o mesmo estado várias vezes.